

Szegedi SZC Vasvári Pál Gazdasági és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

506131203

PROJEKTFELADAT

Virágátrium alkalmazás és webshop fejlesztői dokumentációja

Készítette:

Harmat Balázs, Hódi Kristóf, Orosz Levente

13.b

Szeged

2026.

Tartalomjegyzék

1.	Rendszerarchitektúra áttekintése.....	3
2.	Frontend – React TypeScript + Vite.....	3
3.	Backend – Node.js + Express + MySQL	3
4.	Termékek (/api/products).....	4
5.	Kategóriák (/api/categories).....	5
6.	Felhasználók (/api/users) (/api/login, register).....	5
7.	Rendelt termékek (/api/order-items)	5
8.	Rendelések (/api/orders).....	5
9.	Üzenetek (/api/messages)	6
10.	Adatbázis – MySQL Docker konténerben	7
11.	Admin felület – JavaFX	10
12.	Fejlesztői eszközök	11
13.	Továbbfejlesztési irány – Mobilalkalmazás	11
14.	Irodalomjegyzék / Felhasznált AI eszközök	11

1. Rendszerarchitektúra áttekintése

A projekt három különálló, egymással kommunikáló rétegből épül fel: egy böngészőben futó React-alapú webshop frontend, egy Node.js/Express REST API backend, valamint egy JavaFX-ben megírt asztali adminisztrációs felület. Az adatok tárolását egy MySQL adatbázis látja el, amelyet Docker Desktop konténerben futtatunk, ezzel biztosítva a hordozhatóságot és a fejlesztői környezetek közötti konzisztenciát.

2. Frontend – React TypeScript + Vite

A felhasználói felület React 18 és TypeScript kombinációjával készült, a projektváz generálásához és a fejlesztői szerver kezeléséhez Vite-ot alkalmazunk. A Vite választásának oka az ESM-alapú gyors HMR (Hot Module Replacement), ami drasztikusan lecsökkenti az újrafordítási időt a Create React App-hoz képest.

A TypeScript használata kötelező típusbiztonságot vezet be, így az API-válaszokból érkező adatstruktúrák (pl. Product, Order, Category) interfészként definiálhatók, és a fordítás idején elkaphatók a típushibák. A komponensek funkcionálisak, a state kezelése React Hook-okkal (useState, useEffect) történik. Az API-hívások, axios segítségével kommunikálnak a backend felé.

Az alkalmazás főbb moduljai: terméklista- és termékoldal-nézet, kosár- és pénztár folyamat, felhasználói autentikáció (bejelentkezés/regisztráció), saját profil oldal, üzenetküldési lehetőség.

3. Backend – Node.js + Express + MySQL

A backend Node.js és Express.js alapon épül fel, RESTful API-t szolgáltat a webshop frontend és a JavaFX admin felület számára egyaránt. Az útvonalak logikailag erőforrások szerint vannak csoportosítva, minden modul saját fájlban kap helyet (data/products.js, data/orders.js stb.), amelyeket az app.js regisztrál egy közös /api prefix alá. Az adatbázis- műveletek egy külön routes/... rétegben találhatóak, az Express kontrollerek csak a kérés- válasz ciklust kezelik.

4. Termékek (/api/products)

A termék-végpontok teljes CRUD-ot valósítanak meg. Lekérdezhető az összes termék listája, illetve egyetlen termék azonosító alapján. Új termék felvitele POST kéréssel történik, a módosítás PUT metódussal, a törlés DELETE-tel. Ezeket a végpontokat a JavaFX admin felület használja elsősorban, de a webshop frontend is olvassa a termékadatokat.

5. Kategóriák (/api/categories)

A kategória-végpontok szintén teljes CRUD-ot fednek le, felépítésük analóg a termékekével. Kategória létrehozása, lekérdezése (összes, illetve egy adott), módosítása és törlése érhető el.

A termékek és kategóriák között egy-több kapcsolat áll fenn, a termék rekord tartalmaz egy idegen kulcsot a kategória táblára.

6. Felhasználók (/api/users) (/api/login, register)

A felhasználó-kezelés autentikációs végpontokra korlátozódik. A regisztrációs endpoint fogadja az új felhasználó adatait, a jelszót bcrypt segítségével hashelve tárolja az adatbázisban. A bejelentkezési endpoint ellenőrzi a megadott hitelesítő adatokat, sikeres azonosítás esetén JWT tokenet állít elő és küld vissza a kliensnek. Ezt a tokenet a frontend minden védett kérés Authorization fejlécébe elhelyezi, a backend middleware pedig minden beérkező kérésnél ellenőrzi az érvényességét. Tartalmazza még egy felhasználónak az adatainak a lekérését illetve, összes felhasználónak is a lekérését.

7. Rendelt termékek (/api/order-items)

Az order items modul egyetlen végpontot tartalmaz: az összes rendeléstétel lekérdezését. Ez az adminisztráció számára nyújt áttekintést arról, hogy összesítve milyen termékek és milyen mennyiségben szerepelnek a leadott rendelésekben. Az egyes tételek a rendelés azonosítóján keresztül kapcsolódnak a rendelés táblához.

8. Rendelések (/api/orders)

A rendelés-végpontok teljes CRUD-ot biztosítanak: új rendelés létrehozása, az összes rendelés lekérdezése (admin nézet), egyedi rendelés lekérdezése azonosító alapján, rendelés adatainak módosítása (pl. státuszváltás: „feldolgozás alatt” → „kiszállítva”), valamint rendelés törlése. Ezen felül elérhető egy felhasználó-specifikus végpont is, amely egy adott felhasználó összes

rendelését adja vissza — ezt a webshop frontend rendelési előzmények oldala hívja meg a bejelentkezett felhasználó token-jéből kinyert azonosítóval.

9. Üzenetek (/api/messages)

Az üzenet-modul az admin és a vásárló közötti kétirányú kommunikációt valósítja meg. Egy adott párbeszéd teljes előzménye lekérdezhető a két fél azonosítója alapján — ez adja a csevegésszerű megjelenítés alapját mindkét felületen. Új üzenet küldése POST kéréssel történik, a rekord tartalmazza a küldő és a fogadó azonosítóját, az üzenet szövegét, valamint az időbélyeget. PUT kéréssel meg lehet módosítani az üzenetek státuszát (el van-e olvasva)

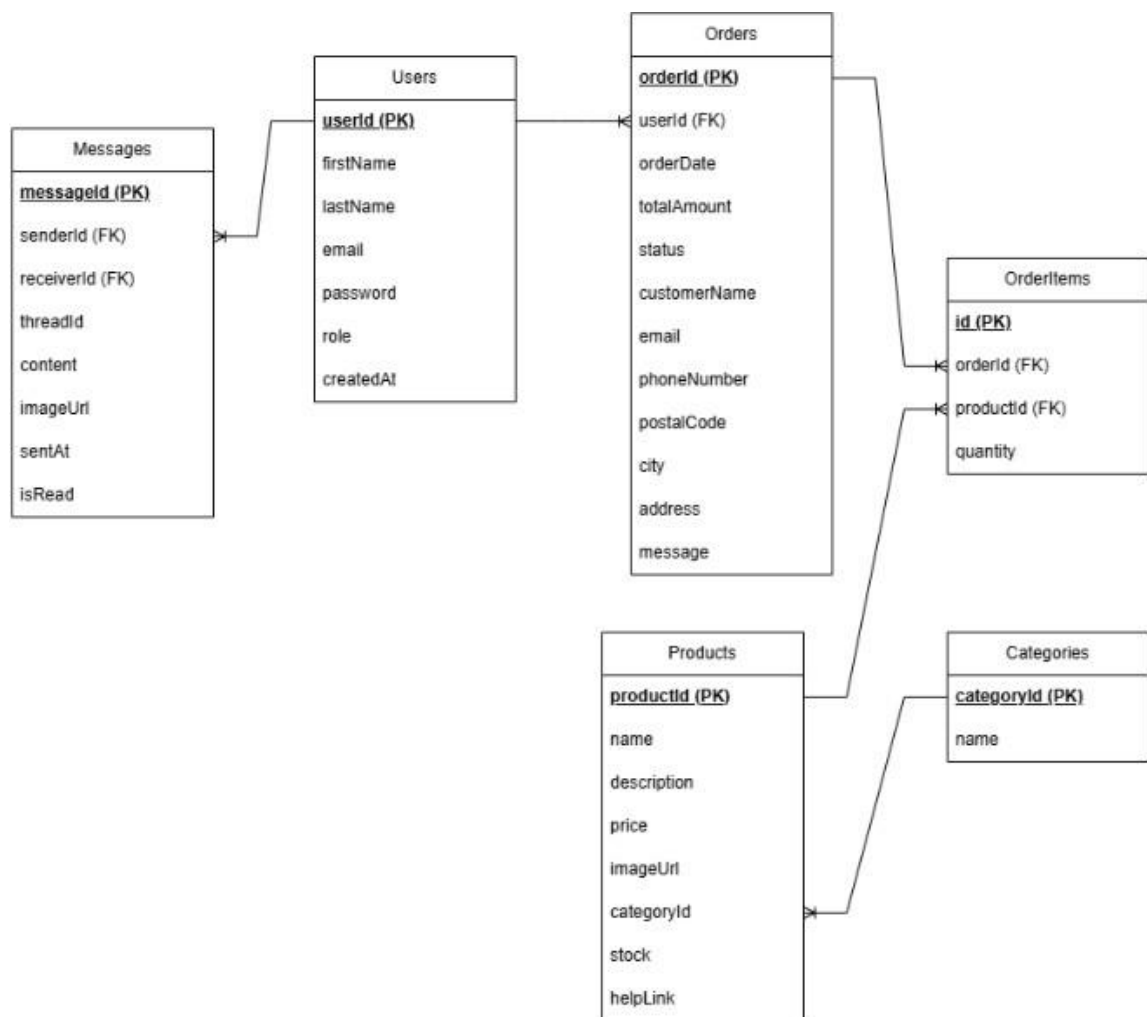
10. Adatbázis – MySQL Docker konténerben

Az adatbázis Docker Desktop segítségével konténerizált MySQL példányban fut. A docker-compose.yml fájl definiálja a konténer konfigurációját: a portmappinget (alapértelmezetten 3306:3306), a perzisztens volume csatolását (hogyan az adatok újraindítás után is

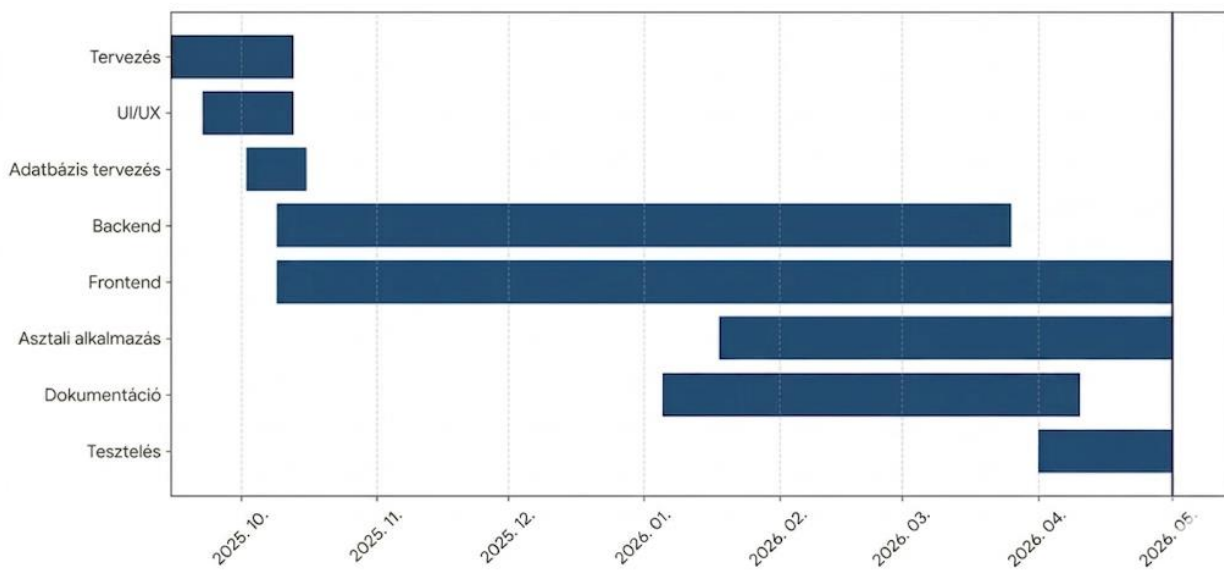
megmaradjanak), valamint a kezdeti környezeti változókat (MYSQL_ROOT_PASSWORD, MYSQL_DATABASE).

Az adatbázis sémáját migrációs szkriptek vagy egy inicializáló SQL fájl tartalmazza, amelyet a konténer első indulásakor futtat le a MySQL. A fejlesztői környezet így egyetlen docker compose up paranccsal elindítható, nem igényel helyi MySQL telepítést.

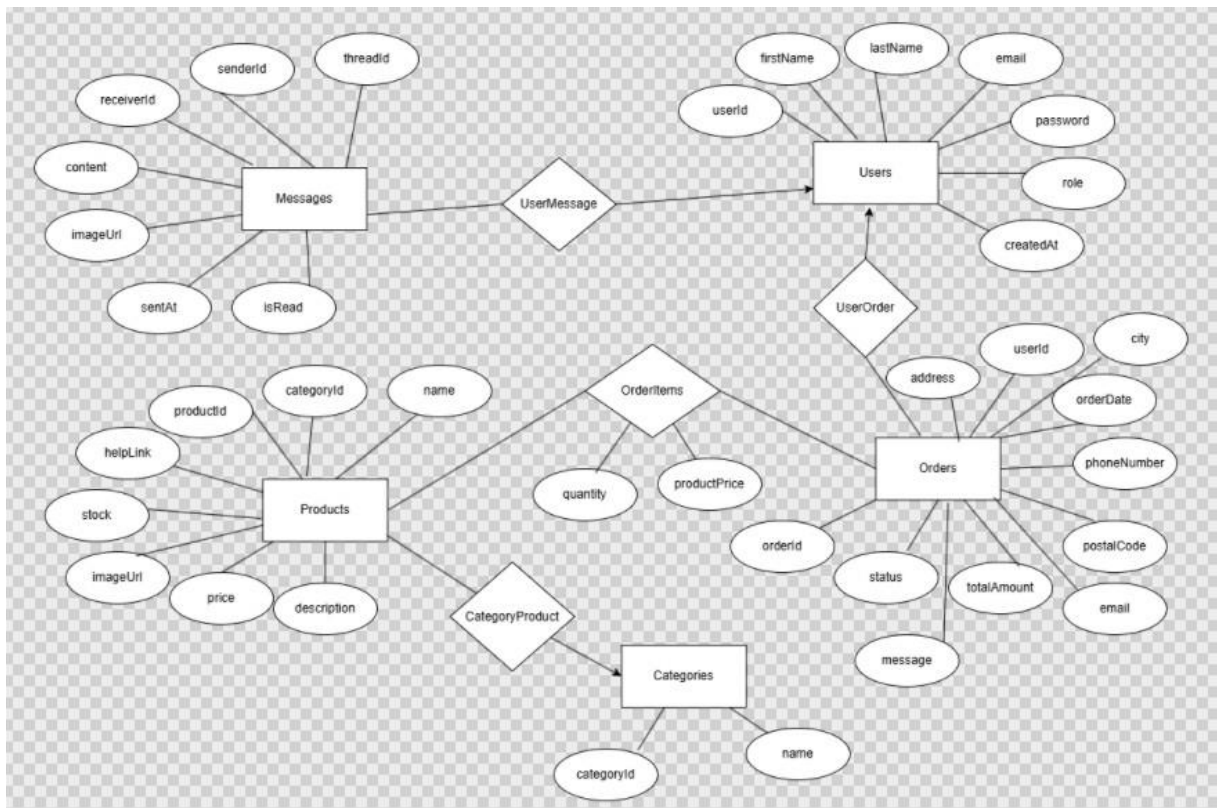
Adatbázis modellek és ábrák: Az adatbázis logikai szerkezetét Egyed-Kapcsolat (E-K) diagram írja le, amely az egyedeket és azok kapcsolatait (egy-több, több-több) ábrázolja. A fizikai adatszerkezetet a Bachmann-diagram szemlélteti, amely a konkrét táblastruktúrát, az elsődleges és idegen kulcsokat, valamint a kapcsolótáblákat tartalmazza.



Bachmann-ábra



Gantt diagramm



Egyed-kapcsolati diagramm

11. Admin felület – JavaFX

Az üzlet munkatársai számára egy különálló, asztali Java alkalmazás készül JavaFX keretrendszerrel. Ez az alkalmazás közvetlenül a backendhez csatlakozik HTTP kéréseken keresztül (a Java beépített HttpClient-jével), így az üzleti logika nem duplikálódik: az admin felület ugyanazokat az API végpontokat hívja, mint a webshop.

Az admin modul főbb funkciói: termékek felvitele, szerkesztése és törlése; rendelések kezelése és állapotváltása (pl. „feldolgozás alatt” → „kiszállítva”); készletszintek monitorozása; kategória-kezelés.

12. Fejlesztői eszközök

Eszköz	Szerep
Visual Studio Code	Frontend (React/TS) és backend (Node.js) fejlesztés
Draw.io	Adatkapcsolódási diagrammok rajzolása
IntelliJ IDEA	JavaFX admin alkalmazás fejlesztése
Docker Desktop	MySQL adatbázis konténerizálása
Google Docs / Word	Dokumentáció készítése

13. Továbbfejlesztési irány – Mobilalkalmazás

A rendszer architektúrája tudatosan API-first szemléletet követ, ami megkönnyíti a jövőbeli mobilalkalmazás integrációját. A tervezett mobilkliens (React Native vagy Flutter) ugyanazokat a REST végpontokat fogja felhasználni, amelyeket a webshop is használ, kiegészítve mobilspecifikus funkciókkal, mint a push értesítések (rendelési státusz változásokor) és esetlegesen helymeghatározás-alapú funkciók.

14. Irodalomjegyzék / Felhasznált AI eszközök

A fejlesztés során az alábbi mesterséges intelligencia alapú eszközök kerültek alkalmazásra, hibakeresésre és dokumentáció-írásra:

- **ChatGPT** – GPT 5.3 modell (OpenAI)
- **Claude** – Sonnet 4.6 modell (Anthropic)
- **DeepSeek** – DeepSeek V3.2 modell (DeepSeek AI)
- **GitHub Copilot** – V1.388.0 (Microsoft / GitHub)